

计算机程序设计基础(2)

--- C++程序设计(4)

孙甲松

sunjiasong@tsinghua.edu.cn

电子工程系 信息认知与智能系统研究所

罗姆楼6-104

电话: 62796193/13901216180

2020. 3.

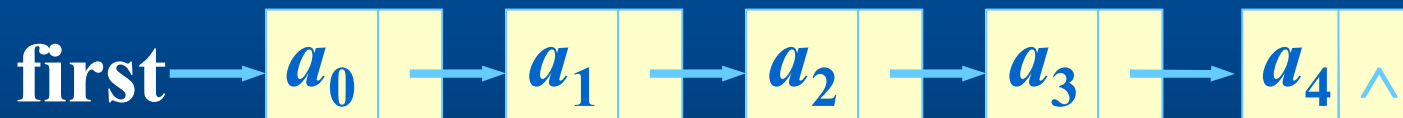
类的应用实例：单链表封装

单链表特点：

- ◆ 每个元素(表项)由结点(***Node***)构成

data	next
-------------	-------------

- ◆ 线性结构



- ◆ 结点可以不连续存储
- ◆ 表可扩充

单链表的类定义

用两个类表达一个概念(单链表)

- ◆ 链表结点(**ListNode**)类
- ◆ 链表(**List**)类

定义方式

- ◆ 复合方式
- ◆ 嵌套方式
- ◆ 继承方式(后面再讲)

```
class List; // 向前引用说明
           //链表类定义（复合方式）
class ListNode {           //链表结点类
public:
    friend class List;     //链表类为其友元类
private:
    int data;              //结点数据, 整型
    ListNode *next;        //结点指针
};

class List {               //链表类
private:
    ListNode *first, *current; //表头指针
                                //当前指针
};
```

```
    // 链表类定义(嵌套方式)
class List {
public:
    //链表操作函数.....
private:
    class ListNode {    //嵌套链表结点类
    public:
        int  data;
        ListNode *next;
    };
    ListNode *first, *current; //表头指针,当前指针
};
```

```
int List :: Insert( int x, int i )
{ // 在链表第 i 个结点处插入新元素 x
  ListNode *p = current;
  current = first;
  for ( int k = 0; k < i - 1; k++ ) //找第 i-1个结点
  { if ( current == NULL )
      break;
    else current = current->next;
  }
  if ( current == NULL && first != NULL )
  { cout << "无效的插入位置!\n";
    current = p;
    return 0; // 函数返回值0表示插入不成功
  }
}
```

//创建新结点

```
ListNode *newnode = new ListNode(x, NULL);
```

```
if ( first == NULL || i == 0 ) //插在表前
```

```
{  newnode->next = first;  
    first = current = newnode;
```

```
}
```

```
else          // 插在表中间或表尾
```

```
{  newnode->next = current->next;  
    current = current->next = newnode;
```

```
}
```

```
return 1;  // 函数返回值1表示插入成功
```

```
}
```

```
int List :: Remove( int &x, int i )
{ //在链表中删除第 i 个结点, 通过x返回其值
  ListNode *p, *q;
  if ( i == 0 )          //删除表中第 1 个结点
  { q = first;
    current = first = first->next;
  }
  else
  { p = current; current = first;
    //找第 i-1 个结点
    for (int k = 0; k < i-1; k++ )
    { if ( current == NULL ) break;
      else current = current->next;
    }
  }
```



```

if (current == NULL || current->next == NULL)
{
    cout << "无效的删除位置!\n";
    current = p;
    return 0; // 函数返回值0表示删除不成功
}
else //删除表中间或表尾元素
{
    q = current->next; //重新链接
    current = current->next = q->next;
}
}
x = q->data; //返回第 i 个结点的值
delete q; //删除链表节点q
return 1; // 函数返回值1表示删除成功
}

```

```

#include <iostream> // 可运行的完整程序
using namespace std;
class List;          // 链表类定义（复合方式）
class ListNode       // 链表结点类
{public:
    friend class List; // 链表类为其友元类
    ListNode(int x, ListNode *next) //构造函数
    { this->data = x;    this->next = next;
    }
private:
    int data;          //结点数据, 整型
    ListNode *next;    //结点指针
};
class List //链表类
{public:
    List( ) {first = current = NULL; } //构造函数
    int Insert(int x, int i);
    int Remove(int &x, int i);
    void Print( );
private:
    ListNode *first, *current; //表头指针, 当前指针
};

```

```

int List :: Insert(int x, int i) //在链表第 i 个结点处插入新元素 x
{
    ListNode * p = current;  current = first;
    for ( int k = 0; k < i - 1; k++ ) //找第 i-1 个结点
    {
        if ( current == NULL )
            break;
        else current = current->next;
    }
    if ( current == NULL && first != NULL )
    {
        cout << "无效的插入位置!\n";
        current = p;
        return 0;
    }
    ListNode *newnode = new ListNode(x, NULL);
    if ( first == NULL || i == 0 ) //插在表前
    {
        newnode->next = first;
        first = current = newnode;
    }
    else //插在表中间或表尾
    {
        newnode->next = current->next;
        current = current->next = newnode;
    }
    return 1;
}

```

```

int List::Remove(int &x, int i)
{ //在链表中删除第 i 个结点, 通过x返回其值
  ListNode *p, *q;
  if ( i == 0 )          //删除表中第 1 个结点
  { q = first; current = first = first->next; }
  else
  { p = current; current = first;
    //找第 i-1 个结点
    for ( int k = 0; k < i-1; k++ )
    { if ( current == NULL ) break;
      else current = current->next;
    }
    if ( current == NULL || current->next == NULL )
    { cout << "无效的删除位置!\n";
      current = p;
      return 0;
    }
    else          //删除表中间或表尾元素
    { q = current->next;    //重新链接
      current = current->next = q->next;
    }
  }
  x = q->data;      // 返回第 i 个结点的值
  delete q;         // 删除链表节点q
  return 1;
}

```

```

void List::Print( )
{
    ListNode *p=first;
    while (p)
    {
        cout << p->data << ',';
        p=p->next;
    }
    cout << endl;
}

void main( )
{
    List A;
    int x, i;
    for (i=0; i<10; i++)
        A.Insert(i, 0); // 每个元素插在链表头部
    A.Print( );
    for (i=0; i<10; i++)
    {
        if (A.Remove(x, 0)) // 删除链表第一个元素
            cout<<x<<',';
    }
    cout << endl;
}

```

运行结果:

9,8,7,6,5,4,3,2,1,0,

9,8,7,6,5,4,3,2,1,0,

请按任意键继续.....

若把 `if (A.Remove(x, 0))`

改为: `if (A.Remove(x, 1))`

运行结果:

9,8,7,6,5,4,3,2,1,0,

8,7,6,5,4,3,2,1,0,无效的删除位置!

请按任意键继续...

还可以用List对象指针实现一个链表:

```
void main( )
```

```
{ List *A=new List; // 创建一个List动态内存对象
```

```
    int x, i;
```

```
    for (i=0; i<10; i++)
```

```
        A->Insert(i, 0);
```

```
    A->Print( );
```

```
    for (i=0; i<10; i++)
```

```
    {    if(A->Remove(x, 0))
```

```
        cout << x << ',';
```

```
    }
```

```
    cout << endl;
```

```
    delete A;
```

```
}
```

运行结果:

9,8,7,6,5,4,3,2,1,0,

9,8,7,6,5,4,3,2,1,0,

请按任意键继续...

4. 操作符重载

- 操作符重载（operator overloading）是指C++允许将原来的操作符加入新的含义，方便程序员编程。
- 要重载的操作符必须是已有的，不能自己定义（臆造）新的操作符，比如，用**表示幂指数。
- 操作符重载既不能改变原运算符的运算优先级和结合性，也不能改变操作数的个数（变多或变少）。
- 可重载的操作符有45个，都是一目、二目操作符，但"."、".*"、"::"、"?:"、sizeof 以及 typeid 这6个操作符不能重载，“否则会带来难以琢磨的问题”。
- 需要为操作符新加入的每一个功能编写一个操作符函数，而且只能是类类型的重载操作符，操作数中至少有一个是自定义的类类型。

4. 操作符重载 (续1)

- 实际上编译系统是将指定的运算表达式转化为对运算符函数的调用，运算对象转化为运算符函数的实参。
- 编译系统对重载运算符的选择，沿用函数重载的选择原则，即操作对象的类型决定操作符的语义。
- 例如：A+B
 - (1) 当A，B为int, 转化为整型数加汇编语句；
 - (2) 当A，B为double, 转化为浮点数加汇编语句；
 - (3) 当A，B为类C类型时, 转化为调用C类中的“+”操作符函数，A、B为其实参；
 - (4) 当A，B为类D类型时, 转化为调用D类中的“+”操作符函数，A、B为其实参。

4. 操作符重载 (续2)

- 操作符重载的步骤是：
- a. 生成一个类，定义重载操作符的操作数据类型；
- b. 在public中，包含一个操作符友元函数或一个操作符成员函数；
- c. 在程序中，定义一个操作符函数(相应的操作符友元函数或成员函数)。
- 例如：求幂指数 10^3 。操作符 $^$ 的原意是异或操作， $10^3=9$ 。我们将使 $^$ 成为对某一个数的乘幂，即 10^3 结果为1000。

4. 操作符重载 (续3)

```
#include <iostream>
using namespace std;
class pwr {
    public:
        friend int operator ^(pwr, pwr);
        int num;
};
int operator ^(pwr b, pwr e)
{ int t, temp;
  temp = b.num;
  for (t=e.num-1; t; t--)
      temp *= b.num;
  return temp ;
}
```

4. 操作符重载 (续4)

```
void main()  
{ pwr b, e;  
  b.num=10;  
  e.num = 3;  
  int r1, r2;  
  r1 = b^e;  // b^e 是求be的值  
  r2 = 10^3; // 10^3是求10与3的异或  
  cout << r1 << "  " << r2 << endl;  
}
```

输出结果: 1000 9

以上是最简单的写法，只需要一点有关类的知识。若将num定义为私有成员，加入构造函数，程序重写为：

4. 操作符重载 (续5)

```
#include <iostream>
using namespace std;
class pwr {
    public:    pwr(int i) { num=i; }
              friend int operator ^(pwr, pwr);
    private: int num;
};
int operator ^(pwr b, pwr e)
{ int t, temp;
  temp = b.num;
  for (t=e.num-1; t; t--)
      temp *= b.num;
  return temp ;
}
```

4. 操作符重载 (续6)

```
void main( )  
{ pwr b(10), e(3); //再次回味C++的赋初值  
  int r1, r2;  
  r1 = b^e; // b^e 是求be的值  
  r2 = 10^3; // 10^3是求10与3的异或  
  cout << r1 << " " << r2 << endl;  
}
```

输出结果: 1000 9

4. 运算符重载 (续7)

- 运算符重载函数有两种形式:

- a. 重载为类的成员函数
- b. 重载为类的友元函数

统称为运算符函数。其书写格式是:

- 函数类型 operator 运算符(形参)

```
{ .....  
}
```

- 重载为成员函数时, 形参个数是运算符操作数个数减1 (但后置的++、--除外)。
- 重载为友元函数时, 形参个数与运算符操作数个数相同。

4. 操作符重载 (续8)

操作符函数按照操作符来分，分为：

- 单目（一元）操作符
- 二目（二元）操作符

操作符的目（元）数不同，操作符函数形式也不同。

我们分别讲以下内容：

- 4.1 二目操作符的成员函数
- 4.2 二目操作符的友元函数
- 4.3 单目操作符的成员函数
- 4.4 单目操作符的友元函数
- 4.5 特殊操作符的重载
- 4.6 << 操作符的重载
- 4.7 类型操作符的重载 (double) ---类型转换函数
- 4.8 重载操作符执行时参数的传递
- 4.9 重载++,--操作符函数的返回值

4.1 二目操作符的成员函数

- 若二目操作符是B，其操作对象分别是：

op1、op2，则表达式应该为: op1 B op2

其中op1为A类对象，则B应该重载为A类的成员函数，形参类型是op2所属的类型。

即表达式: op1 B op2 等价于:

op1.operator B(op2) 函数调用。

例如：将“+”、“-”算术运算符重载为能对复数类进行相应运算。实部和虚部分别相加减。

- 函数类型 operator 操作符（形参）{ }

其中的函数类型、形参都是复数类类型。

```

#include <iostream>
using namespace std;
class complex {    //复数类声明
public:            //外部接口
    complex(double r=0.0, double i=0.0) : r(r), i(i)
    { cout << "Constructor! "; display( ); }
    complex(complex &c) : r(c.r), i(c.i)
    { cout << "Copy Constructor!"; display( ); }
    ~complex( ) { cout << "Destructor! "; display( ); }
    complex operator + (complex c2); //运算符+重载成员函数
    complex operator - (complex c2); //运算符-重载成员函数
    void display( );    //输出复数
private:            //私有数据成员
    double r;    //复数实部
    double i;    //复数虚部
};

```

```

complex complex::operator +(complex c2) //重载运算符函数实现
{ return complex(r+c2.r, i+c2.i); } //创建临时无名对象作为返回值
complex complex::operator -(complex c2)//重载运算符函数实现
{ return complex(r-c2.r, i-c2.i); } //创建临时无名对象作为返回值
void complex::display( )
{ cout<<"("<<r<<","<<i<<)"<<endl; }
void main( ) //主函数
{ complex c1(5,4),c2(2,10),c3; //声明复数类的对象
  cout<<"c1="; c1.display( );
  cout<<"c2="; c2.display( );
  c3=c1-c2; //使用重载运算符完成复数减法
  cout<<"c3=c1-c2=";
  c3.display( );
  c3=c1+c2; //使用重载运算符完成复数加法
  cout<<"c3=c1+c2=";
  c3.display( );
}

```

运行结果:

Constructor! (5,4)

Constructor! (2,10)

Constructor! (0,0)

c1=(5,4)

c2=(2,10)

Copy Constructor!(2,10)

Constructor! (3,-6)

Destructor! (2,10)

Destructor! (3,-6)

c3=c1-c2=(3,-6)

Copy Constructor!(2,10)

Constructor! (7,14)

Destructor! (2,10)

Destructor! (7,14)

c3=c1+c2=(7,14)

Destructor! (7,14)

Destructor! (2,10)

Destructor! (5,4)

请按任意键继续...

4.1 二目操作符的成员函数(续1)

重载运算符函数也可以写成：(可能这个更好理解)

```
complex complex::operator +(complex c2) //重载运算符函数+实现
```

```
{ complex t; //创建一个临时对象,初值 (0, 0)
```

```
    t.r = r + c2.r;
```

```
    t.i = i + c2.i;
```

```
    return t; //临时对象作为返回值
```

```
}
```

```
complex complex::operator -(complex c2) //重载运算符函数-实现
```

```
{ complex t; //创建一个临时对象,初值 (0, 0)
```

```
    t.r = r - c2.r;
```

```
    t.i = i - c2.i;
```

```
    return t; //临时对象作为返回值
```

```
}
```

运行结果:

```
Constructor! (5,4)
Constructor! (2,10)
Constructor! (0,0)
c1=(5,4)
c2=(2,10)
Copy Constructor!(2,10)
Constructor! (0,0)
Copy Constructor!(3,-6)
Destructor! (3,-6)
Destructor! (2,10)
Destructor! (3,-6)
c3=c1-c2=(3,-6)
Copy Constructor!(2,10)
Constructor! (0,0)
Copy Constructor!(7,14)
Destructor! (7,14)
Destructor! (2,10)
Destructor! (7,14)
c3=c1+c2=(7,14)
Destructor! (7,14)
Destructor! (2,10)
Destructor! (5,4)
请按任意键继续...
```

修改前的运行结果:

```
Constructor! (5,4)
Constructor! (2,10)
Constructor! (0,0)
c1=(5,4)
c2=(2,10)
Copy Constructor!(2,10)

Constructor! (3,-6)

Destructor! (2,10)
Destructor! (3,-6)
c3=c1-c2=(3,-6)
Copy Constructor!(2,10)

Constructor! (7,14)

Destructor! (2,10)
Destructor! (7,14)
c3=c1+c2=(7,14)
Destructor! (7,14)
Destructor! (2,10)
Destructor! (5,4)
请按任意键继续...
```

4.1 二目操作符的成员函数(续2)

重载运算符函数还可以写成:

```
complex complex::operator +(complex c2) //重载运算符函数+实现
{
    complex t(r+c2.r, i+c2.i); //创建一个临时对象
    return t; //临时对象作为返回值
}

complex complex::operator -(complex c2) //重载运算符函数-实现
{
    complex t(r-c2.r, i-c2.i); //创建一个临时对象
    return t; //临时对象作为返回值
}
```

这个只是上一个的书写简单化，但减少了操作次数，执行效率会更高一些。但比不上直接返回无名对象效率高。

运行结果:

```
Constructor! (5,4)
Constructor! (2,10)
Constructor! (0,0)
c1=(5,4)
c2=(2,10)
Copy Constructor!(2,10)
Constructor! (3,-6)
Copy Constructor!(3,-6)
Destructor! (3,-6)
Destructor! (2,10)
Destructor! (3,-6)
c3=c1-c2=(3,-6)
Copy Constructor!(2,10)
Constructor! (7,14)
Copy Constructor!(7,14)
Destructor! (7,14)
Destructor! (2,10)
Destructor! (7,14)
c3=c1+c2=(7,14)
Destructor! (7,14)
Destructor! (2,10)
Destructor! (5,4)
请按任意键继续...
```


能否进一步减少函数调用，提高执行效率呢？

可以！把重载运算符函数参数改为引用方式！

```
#include <iostream>
using namespace std;
class complex {          //复数类声明
public: //外部接口
    complex(double r=0.0, double i=0.0) : r(r), i(i)
    { cout << "Constructor! "; display( ); }
    complex(complex &c) : r(c.r), i(c.i)
    { cout << "Copy Constructor!"; display(); }
    ~complex( ) { cout << "Destructor! "; display( ); }
    complex operator + (complex &c2) //函数参数引用方式
    { return complex(r+c2.r, i+c2.i); }
    complex operator - (complex &c2) //函数参数引用方式
    { return complex(r-c2.r, i-c2.i); }
    void display( )          //输出复数
    { cout<<"("<<r<<","<<i<<")"<< endl; }
private: //私有数据成员
    double r; //复数实部
    double i; //复数虚部
}; // main函数与前面相同，不再写出
```

运行结果:

```
Constructor! (5,4)
Constructor! (2,10)
Constructor! (0,0)
c1=(5,4)
c2=(2,10)
Constructor! (3,-6)
Destructor! (3,-6)
c3=c1-c2=(3,-6)
Constructor! (7,14)
Destructor! (7,14)
c3=c1+c2=(7,14)
Destructor! (7,14)
Destructor! (2,10)
Destructor! (5,4)
请按任意键继续...
```

原来最好的运行结果:

```
Constructor! (5,4)
Constructor! (2,10)
Constructor! (0,0)
c1=(5,4)
c2=(2,10)
Copy Constructor!(2,10)
Constructor! (3,-6)
Destructor! (2,10)
Destructor! (3,-6)
c3=c1-c2=(3,-6)
Copy Constructor!(2,10)
Constructor! (7,14)
Destructor! (2,10)
Destructor! (7,14)
c3=c1+c2=(7,14)
Destructor! (7,14)
Destructor! (2,10)
Destructor! (5,4)
请按任意键继续...
```

若害怕引用方式带来的副作用，可以加上const修饰符，把：

```
complex operator + (complex &c2) //参数引用方式
```

```
{    return complex(r+c2.r, i+c2.i); }
```

```
complex operator - (complex &c2) //参数引用方式
```

```
{    return complex(r-c2.r, i-c2.i); }
```

改为：

```
complex operator + (const complex &c2) //常量参数引用方式
```

```
{    return complex(r+c2.r, i+c2.i); }
```

```
complex operator - (const complex &c2) //常量参数引用方式
```

```
{    return complex(r-c2.r, i-c2.i); }
```

这样在成员操作符函数+和-中将不能修改所引用的对象c2。

4.2 二目操作符的友元函数

- 若二目操作符是B，其操作对象分别是：
op1、op2，则表达式应该为: op1 B op2

其中op1、op2都为A类对象，将操作符B重载为A类的友元函数，使之能访问该类的私有成员。

表达式: op1 B op2等价于: operator B(op1,op2)的函数调用。

例如：将“+”、“-”算术运算符重载为能对复数类进行相应运算。实部和虚部分别相加减。

```

#include <iostream>
using namespace std;
class complex {    //复数类声明
public:            //外部接口
    complex(double r=0.0,double i=0.0) : r(r), i(i)
    { cout << "Constructor! "; display( ); }
    complex(complex &c) : r(c.r), i(c.i)
    { cout << "Copy Constructor!"; display( );}
    ~complex( ) { cout << "Destructor! "; display( ); }
    friend complex operator + (complex c1,complex c2);
                                //运算符+重载友元函数
    friend complex operator - (complex c1,complex c2);
                                //运算符-重载友元函数
    void display( ) { cout<< "("<<r<<","<<i<< ")"<<endl; }
private:           //私有数据成员
    double r;    //复数实部
    double i;    //复数虚部
};

```

```
complex operator +(complex c1,complex c2) //重载运算符函数实现
{ return complex(c1.r+c2.r, c1.i+c2.i); } //创建临时无名对象返回
complex operator -(complex c1,complex c2) //重载运算符函数实现
{ return complex(c1.r-c2.r, c1.i-c2.i); } //创建临时无名对象返回
void main( ) //主函数
{ complex c1(5,4), c2(2,10), c3; //声明复数类的对象
  cout<<"c1="; c1.display( );
  cout<<"c2="; c2.display( );
  c3=c1-c2; //使用重载运算符完成复数减法
  cout<<"c3=c1-c2="; c3.display( );
  c3=c1+c2; //使用重载运算符完成复数加法
  cout<<"c3=c1+c2="; c3.display( );
}
```

友元函数运行结果:

```
Constructor! (5,4)
Constructor! (2,10)
Constructor! (0,0)
c1=(5,4)
c2=(2,10)
Copy Constructor!(2,10)
Copy Constructor!(5,4)
Constructor! (3,-6)
Destructor! (5,4)
Destructor! (2,10)
Destructor! (3,-6)
c3=c1-c2=(3,-6)
Copy Constructor!(2,10)
Copy Constructor!(5,4)
Constructor! (7,14)
Destructor! (5,4)
Destructor! (2,10)
Destructor! (7,14)
c3=c1+c2=(7,14)
Destructor! (7,14)
Destructor! (2,10)
Destructor! (5,4)
请按任意键继续...
```

成员函数运行结果:

```
Constructor! (5,4)
Constructor! (2,10)
Constructor! (0,0)
c1=(5,4)
c2=(2,10)
Copy Constructor!(2,10)
Constructor! (3,-6)
Destructor! (2,10)
Destructor! (3,-6)
c3=c1-c2=(3,-6)
Copy Constructor!(2,10)
Constructor! (7,14)
Destructor! (2,10)
Destructor! (7,14)
c3=c1+c2=(7,14)
Destructor! (7,14)
Destructor! (2,10)
Destructor! (5,4)
请按任意键继续...
```

4.2 二目操作符的友元函数(续1)

重载运算符 “+”友元函数也可以写成:

```
complex operator +(complex c1, complex c2)
{ complex t; //创建一个临时对象,初值 (0, 0)
  t.r = c1.r + c2.r;
  t.i = c1.i + c2.i;
  return t; //临时对象作为返回值
}
```

或:

```
complex operator +(complex c1, complex c2)
{ complex t(c1.r + c2.r, c1.i + c2.i); //创建一个临时对象
  return t; //临时对象作为返回值
}
```


友元函数2运行结果:

```
Constructor! (5,4)
Constructor! (2,10)
Constructor! (0,0)
c1=(5,4)
c2=(2,10)
Copy Constructor!(2,10)
Copy Constructor!(5,4)
Constructor! (3,-6)
Copy Constructor!(3,-6)
Destructor! (3,-6)
Destructor! (5,4)
Destructor! (2,10)
Destructor! (3,-6)
c3=c1-c2=(3,-6)
Copy Constructor!(2,10)
Copy Constructor!(5,4)
Constructor! (7,14)
Copy Constructor!(7,14)
Destructor! (7,14)
Destructor! (5,4)
Destructor! (2,10)
Destructor! (7,14)
c3=c1+c2=(7,14)
Destructor! (7,14)
Destructor! (2,10)
Destructor! (5,4)
请按任意键继续...
```

友元函数运行结果:

```
Constructor! (5,4)
Constructor! (2,10)
Constructor! (0,0)
c1=(5,4)
c2=(2,10)
Copy Constructor!(2,10)
Copy Constructor!(5,4)
Constructor! (3,-6)
Destructor! (5,4)
Destructor! (2,10)
Destructor! (3,-6)
c3=c1-c2=(3,-6)
Copy Constructor!(2,10)
Copy Constructor!(5,4)
Constructor! (7,14)
Destructor! (5,4)
Destructor! (2,10)
Destructor! (7,14)
c3=c1+c2=(7,14)
Destructor! (7,14)
Destructor! (2,10)
Destructor! (5,4)
请按任意键继续...
```

能否进一步减少函数调用，提高执行效率呢？

能！引用方式

```
#include <iostream>
using namespace std;
class complex {    //复数类声明
public:            //外部接口
    complex(double r=0.0,double i=0.0) : r(r), i(i)
    { cout << "Constructor! "; display( ); }
    complex(complex &c) : r(c.r),i(c.i)
    { cout << "Copy Constructor!"; display( );}
    ~complex( ) { cout << "Destructor! "; display( ); }
    friend complex operator + (complex &c1,complex &c2)
    { return complex(c1.r+c2.r, c1.i+c2.i); }
    friend complex operator - (complex &c1,complex &c2)
    { return complex(c1.r-c2.r, c1.i-c2.i); }
    void display( ) { cout<< "("<<r<<","<<i<< ")"<<endl; }
private:          //私有数据成员
    double r;     //复数实部
    double i;     //复数虚部
};
```

引用方式友元函数运行结果:

```
Constructor! (5,4)
Constructor! (2,10)
Constructor! (0,0)
c1=(5,4)
c2=(2,10)
Constructor! (3,-6)
Destructor! (3,-6)
c3=c1-c2=(3,-6)
Constructor! (7,14)
Destructor! (7,14)
c3=c1+c2=(7,14)
Destructor! (7,14)
Destructor! (2,10)
Destructor! (5,4)
请按任意键继续...
```

原来的友元函数运行结果:

```
Constructor! (5,4)
Constructor! (2,10)
Constructor! (0,0)
c1=(5,4)
c2=(2,10)
Copy Constructor!(2,10)
Copy Constructor!(5,4)
Constructor! (3,-6)
Destructor! (5,4)
Destructor! (2,10)
Destructor! (3,-6)
c3=c1-c2=(3,-6)
Copy Constructor!(2,10)
Copy Constructor!(5,4)
Constructor! (7,14)
Destructor! (5,4)
Destructor! (2,10)
Destructor! (7,14)
c3=c1+c2=(7,14)
Destructor! (7,14)
Destructor! (2,10)
Destructor! (5,4)
请按任意键继续...
```

同样若害怕引用方式带来的副作用，可以加上const修饰符，把：

```
friend complex operator + (complex &c1,complex &c2)
```

```
{ return complex(c1.r+c2.r, c1.i+c2.i); }
```

```
friend complex operator - (complex &c1,complex &c2)
```

```
{ return complex(c1.r-c2.r, c1.i-c2.i); }
```

改为：

```
friend complex operator + (const complex &c1, const complex &c2)
```

```
{ return complex(c1.r+c2.r, c1.i+c2.i); }
```

```
friend complex operator - (const complex &c1, const complex &c2)
```

```
{ return complex(c1.r-c2.r, c1.i-c2.i); }
```

这样在友元操作符函数+和-中将不能修改所引用的对象c1， c2

4.3 单目操作符的成员函数

- 若单目操作符是U，其操作对象是op，则表达式应该为: U op 或 op U
- 分别称为：前置单目运算符、后置单目运算符。
- 对于前置单目运算符 U op :

其op为A类对象，将操作符U重载为A类的成员函数，无形参。

表达式: U op等价于: op.operator U() 的函数调用。

- 对于后置单目运算符 op U : （例如 ++,--）

其op为A类对象，将操作符U重载为A类的成员函数，且具有一个int类型的形参。

4.3 单目操作符的成员函数(续1)

表达式: `op U` 等价于:

`op.operator U(0)` 的函数调用。

例如: 将前置 “++” 和后置 “++” 运算符重载为能对时钟类进行相应运算。实现时间增加1秒, 但时钟的时间保持正确的数字格式。

```

#include<iostream>
using namespace std;
class Clock    //时钟类声明
{ public:      //外部接口
    Clock(int H=0, int M=0, int S=0);
    void ShowTime( );
    void operator ++( );    //前置单目运算符重载
    void operator ++(int);  //后置单目运算符重载
private:      //私有数据成员
    int Hour, Minute, Second;
};

Clock::Clock(int H, int M, int S)    //构造函数
{ if(0<=H && H<24 && 0<=M && M<60 && 0<=S && S<60)
    { Hour=H; Minute=M; Second=S; }
  else    cout<<"Time error!"<<endl;
}

```

```

void Clock::ShowTime( )    //显示时间函数
{  cout<<Hour<<":"<<Minute<<":"<<Second<<endl;
}

void Clock::operator ++( )    //前置单目运算符重载函数
{  ++Second;
    if (Second>=60)
    {  Second=Second-60;
        ++Minute;
        if (Minute>=60)
        {  Minute=Minute-60;
            ++Hour;
            Hour=Hour%24;
        }
    }
    cout<<"++Clock: ";
}

```



```
void Clock::operator ++(int) //后置单目运算符重载
{    //注意形参表中的整型参数
    Second++;
    if (Second>=60)
    { Second=Second-60;
      Minute++;
      if (Minute>=60)
      { Minute=Minute-60;
        Hour++;
        Hour=Hour%24;
      }
    }
    cout<<"Clock++: ";
}
```

```
void main( )  
{ Clock myClock(23,59,58);  
  cout<<"Primary time output:";  
  myClock.ShowTime( );  
  myClock++; myClock.ShowTime( );  
  ++myClock; myClock.ShowTime( );  
  myClock++; myClock.ShowTime( );  
  ++myClock; myClock.ShowTime( );  
}
```

运行结果:

Primary time output:23:59:58

Clock++: 23:59:59

++Clock: 0:0:0

Clock++: 0:0:1

++Clock: 0:0:2

4.4 单目操作符的友元函数

- 若单目操作符是U，其操作对象是op，则表达式应该为：U op 或 op U

- 对于前置单目运算符 U op :

其中op为A类对象，将操作符U重载为A类的友元函数，op为其形参。

表达式: U op 等价于: operator U(op) 的函数调用。

- 对于后置单目运算符 op U: (++/--)

其中op为A类对象，将操作符U重载为A类的友元函数，除了op为其形参外，列表中还要增加一个int类型的形参，但不必写形参名。

表达式: op U等价于: operator U(op, 0) 的函数调用

4.4 单目操作符的友元函数(续1)

例如：将前置“++”和后置“++”运算符重载为能对时钟类进行相应运算。实现时间增加1秒，但时钟的时间保持正确的数字格式。用友元函数来实现。

```

#include<iostream>
using namespace std;
class Clock    //时钟类声明
{ public:      //外部接口
    Clock(int H=0, int M=0, int S=0);
    Clock(Clock &p);
    ~Clock( ){ cout << "Destructor! "; ShowTime( ); }
    void ShowTime( );
    friend void operator ++(Clock &c);    //前置单目运算符重载
    friend void operator ++(Clock &c, int); //后置单目运算符重载
private:     //私有数据成员
    int Hour, Minute, Second;
};
Clock::Clock(int H, int M, int S)    //构造函数
{ if(0<=H && H<24 && 0<=M && M<60 && 0<=S && S<60)
    { Hour=H; Minute=M; Second=S; }
  else    cout<<"Time error!"<<endl;
  cout << "Constructor! "; ShowTime( );
}

```

```

void Clock::ShowTime( )    //显示时间函数
{  cout<<Hour<<":"<<Minute<<":"<<Second<<endl;
}

void operator ++(Clock &c)  //前置单目运算符重载函数
{  ++c.Second;
    if (c.Second>=60)
    {  c.Second -= 60;
        ++c.Minute;
        if (c.Minute>=60)
        {  c.Minute -= 60;
            ++c.Hour;
            c.Hour %= 24;
        }
    }
    cout<<"++Clock: ";
}

```

```
void operator ++(Clock &c, int)    //后置单目运算符重载
{                                  //注意形参表中的整型参数
    c.Second++;
    if (c.Second>=60)
    {   c.Second -= 60;
        c.Minute++;
        if (c.Minute>=60)
        {   c.Minute -= 60;
            c.Hour++;
            c.Hour %= 24;
        }
    }
    cout<<"Clock++: ";
}
```

```
Clock::Clock(Clock &p)    //复制构造函数
{
    Hour=p.Hour;
    Minute=p.Minute;
    Second=p.Second;
    cout << "Copy constructor: ";
    ShowTime( );
}

void main( )
{
    Clock myClock(23,59,58);
    cout<< "Primary time output:";
    myClock.ShowTime( );
    myClock++; myClock.ShowTime( );
    ++myClock; myClock.ShowTime( );
    myClock++; myClock.ShowTime( );
    ++myClock; myClock.ShowTime( );
}
```


运行结果:

Constructor! 23:59:58

Primary time output:23:59:58

Clock++: 23:59:59

++Clock: 0:0:0

Clock++: 0:0:1

++Clock: 0:0:2

Destructor! 0:0:2

请按任意键继续...

如果把++操作符的友元函数参数改为非引用方式:

```
#include<iostream>
using namespace std;
class Clock    //时钟类声明
{ public:      //外部接口
    Clock(int H=0, int M=0, int S=0);
    Clock(Clock &p);
    ~Clock( ){ cout << "Destructor! "; ShowTime( ); }
    void ShowTime( );
    friend void operator ++(Clock c);    //前置单目运算符重载
    friend void operator ++(Clock c, int); //后置单目运算符重载
private:     //私有数据成员
    int Hour, Minute, Second;
};
Clock::Clock(int H, int M, int S)    //构造函数
{ if(0<=H && H<24 && 0<=M && M<60 && 0<=S && S<60)
    { Hour=H; Minute=M; Second=S; }
    else    cout<<"Time error!"<<endl;
    cout << "Constructor: "; ShowTime( );
}
```

```

Clock::Clock(Clock &p)    //复制构造函数
{
    Hour=p.Hour; Minute=p.Minute; Second=p.Second;
    cout << "Copy constructor: "; ShowTime();
}

void Clock::ShowTime( )    //显示时间函数
{
    cout<<Hour<<":"<<Minute<<":"<<Second<<endl; }

void operator ++(Clock c)    //前置单目运算符重载函数
{
    ++c.Second;
    if (c.Second>=60)
    {
        c.Second -= 60;
        ++c.Minute;
        if (c.Minute>=60)
        {
            c.Minute -= 60;
            ++c.Hour;
            c.Hour %= 24;
        }
    }
    cout<<"++Clock: ";
}

```

```

void operator ++(Clock c, int)           //后置单目运算符重载
{
    //注意形参表中的整型参数
    c.Second++;
    if (c.Second>=60)
    { c.Second -= 60;    c.Minute++;
      if (c.Minute>=60)
      { c.Minute -= 60;  c.Hour++;  c.Hour %= 24;  }
    }
    cout<<"Clock++: ";
}

void main( )
{ Clock myClock(23,59,58);
  cout<<"Primary time output:";
  myClock.ShowTime( );
  myClock++; myClock.ShowTime( );
  ++myClock; myClock.ShowTime( );
  myClock++; myClock.ShowTime( );
  ++myClock; myClock.ShowTime( );
}

```

运行结果:

Constructor! 23:59:58

Primary time output:23:59:58

clock copy constructor:23:59:58

Clock++: Destructor! 23:59:59

23:59:58

clock copy constructor:23:59:58

++Clock: Destructor! 23:59:59

23:59:58

clock copy constructor:23:59:58

Clock++: Destructor! 23:59:59

23:59:58

clock copy constructor:23:59:58

++Clock: Destructor! 23:59:59

23:59:58

Destructor! 23:59:58

请按任意键继续...

从运行结果可以看到:

对于一元操作符重载函数，
如果不用引用方式，将无法
改变相应对象的值！

因为:

void operator ++(Clock c);

void operator ++(Clock c,int);

这两个函数只是对Clock对象
的一个副本形参进行操作。

Clock对象本身并没有被改变。

第5次作业:

见网络学堂第5次作业

2题必做, 1题选做